Time-Discretization Bias and the Feller Condition in Deep BSDE Pricing of Multi-Asset Options under Heston Stochastic Volatility

Arav Rawat* Advait Kaimal*

October 2026

Abstract

We study the accuracy of the Deep BSDE method for pricing arithmetic basket call options on up to $d = 20$ assets under Heston stochastic volatility, with vol-of-vol $\sigma_v \in [0.1, 0.8]$ spanning both sides of the Feller boundary. Using exact ($d = 1$) and martingale-corrected quadratic-exponential Monte Carlo ($d > 1$) reference prices, we split every pricing error into two parts. The first is the bias of the time-discretized forward dynamics on which the network is trained, which can be measured by plain Monte Carlo. The second is the error of the trained network itself. Across 401 trained solvers, the network error averaged over seeds is small (mean absolute value 0.029% on the main grid, with seed-to-seed standard deviations of at most 0.37%) and does not grow past the Feller boundary. The Euler discretization bias with 40 time steps grows from below 0.05% to 0.69% at $\sigma_v = 0.8$ and accounts for essentially all of the total error. Moving the Feller threshold by changing the mean-reversion speed shows that the bias becomes detectable once the Feller ratio falls below roughly 0.6–0.8, and that the bias divided by σ_v^2 approximately collapses onto a single function of the Feller ratio. The bias decays at about first order in the time step. Of all modifications we tried, only refining the time grid reduces the pricing error; wider or deeper networks, longer training, larger batches, Monte Carlo warm starts and a full-truncation scheme do not. The pricing error decreases with the number of independent assets because the bias does: vol-of-vol effects average out in the basket. With correlated assets this effect disappears and the bias even changes sign. The learned hedge, in contrast, deteriorates markedly with both d and σ_v, with the replication variance reduction falling from 0.993 to 0.57. For this class of problems, the time discretization of the variance process, not the neural network, limits pricing accuracy. We argue that Deep BSDE error budgets should report the two components separately and validate hedges independently of prices.

Keywords: Deep BSDE, Heston model, Feller condition, CIR discretization, basket options, high-dimensional PDEs, neural networks.

MSC 2020: 65C30, 60H35, 91G20, 91G60, 68T07. **JEL:** C63, G13.

1 Introduction

Pricing a European option on a basket of d assets under a stochastic volatility model requires solving a linear parabolic partial differential equation (PDE) in $2d$ spatial variables: one price and one variance coordinate per asset. Grid-based solvers become infeasible beyond a handful of dimensions, which has motivated the Deep BSDE method of E, Han and Jentzen [10, 16]. The method rewrites

*Equal contribution. Independent researchers, Los Angeles, CA, USA. Correspondence: aravrawatcollege@gmail.com.

the PDE as a backward stochastic differential equation (BSDE) through the nonlinear Feynman–Kac formula [11, 24], discretizes it in time, and learns the solution gradient at each time step with a neural network. It has been extended to fully nonlinear equations [6], backward-in-time variants [19], American options [8], quadratic hedging [14], and portfolios of Bermudan options [23]; see [13, 25] for reviews. Convergence results exist under Lipschitz assumptions [15] and, more recently, for forward diffusions with Hölder-$\frac{1}{2}$ coefficients such as the Cox–Ingersoll–Ross (CIR) process [20].

The Heston model [17] is a natural test case because its variance follows a CIR process [9] whose diffusion coefficient $\sigma_v\sqrt{v}$ is not Lipschitz at $v = 0$. By Feller's boundary classification [12], the variance stays strictly positive if and only if the *Feller ratio*

$$\nu \;=\; \frac{2\kappa\theta}{\sigma_v^2} \;\geq\; 1. \tag{1}$$

Calibrations of the Heston model to equity index options frequently violate (1). For example, the Euro Stoxx 50 calibration reported by Albrecher et al. [2] has $\nu \approx 0.38$, and the standard equity test case of Andersen [5] has $\nu = 0.18$. When $\nu < 1$ the variance process touches zero, and simple Euler-type discretizations must be patched (by truncating or reflecting negative variance). These patches are known to introduce substantial bias [3, 5, 18, 22].

Existing Deep BSDE studies with Heston dynamics either use parameters that comfortably satisfy the Feller condition (e.g. $\nu = 5$ in the multi-asset experiments of [14]) or stay in one asset dimension [23]. Neural least-squares Monte Carlo methods have been applied to Bermudan basket options under Heston with up to 50 assets [1]. We are not aware of a systematic study of how Deep BSDE pricing accuracy depends jointly on the number of assets and on the vol-of-vol across the Feller boundary, and in particular of one that separates the two sources of error: *(i)* the bias of the time-discretized forward process that the network is trained on, and *(ii)* the approximation and optimization error of the network itself.

Contributions.

1. We map the pricing error of the Deep BSDE method for multi-asset basket calls under Heston over $d \in \{1, 2, 5, 10, 20\}$ and $\sigma_v \in [0.1, 0.8]$ (401 trained solvers across all experiments), against high-accuracy references: the semi-closed-form Heston price for $d = 1$, and a martingale-corrected quadratic-exponential (QE) Monte Carlo scheme [5] for $d > 1$.

2. We decompose every error into a *discretization bias*, measured independently by large-sample Monte Carlo of the exact same discretized dynamics used in training, and a *solver error*. The solver error is small (typically below 0.1%) and does not grow past the Feller boundary. The discretization bias accounts for essentially all of the degradation.

3. We test whether the Feller ratio controls the degradation by moving the Feller threshold (varying the mean-reversion speed κ). The onset of the bias moves with the threshold and begins at Feller ratios of roughly 0.6–0.8. The bias divided by σ_v^2 approximately collapses onto a single function of the Feller ratio.

4. We show that refining the time grid reduces the error predictably (at roughly first order), whereas wider or deeper networks, longer training, larger batches, Monte Carlo warm-starting and switching to a full-truncation scheme [22] do not.

5. We measure hedge quality directly (initial deltas against pathwise Monte Carlo deltas, and the variance reduction of the learned replication strategy) and study the effect of dimension and inter-asset correlation.

The remainder of the paper is organized as follows. Section 2 states the model and the BSDE. Section 3 describes the solver, discretization schemes, reference prices and error decomposition. Section 4 reports the experiments and Section 5 discusses implications and limitations.

2 Model and BSDE formulation

Multi-asset Heston model. Under the risk-neutral measure, for $i = 1, \dots, d$,

$$dS_i(t) = rS_i(t)\,dt + \sqrt{v_i(t)}\,S_i(t)\,dW_i(t), \tag{2}$$

$$dv_i(t) = \kappa\big(\theta - v_i(t)\big)dt + \sigma_v\sqrt{v_i(t)}\,dB_i(t), \tag{3}$$

with $B_i = \rho W_i + \sqrt{1-\rho^2}\,\widetilde{W}_i$, where $\widetilde{W}_1, \dots, \widetilde{W}_d$ are independent Brownian motions independent of W, and $d\langle W_i, W_j\rangle_t = \rho_S\,dt$ for $i \neq j$. Hence each asset has its own variance process with leverage correlation ρ, and the assets are linked only through ρ_S. Unless stated otherwise $\rho_S = 0$ (independent assets). Throughout, $S_i(0) = 100$, $v_i(0) = \theta = 0.04$, $\kappa = 2$, $\rho = -0.7$, $r = 0.05$, $T = 0.5$ and strike $K = 100$. With these values the Feller threshold is $\sigma_v^* = \sqrt{2\kappa\theta} = 0.4$. We price the arithmetic basket call with payoff

$$g(S(T)) = \Big(\tfrac{1}{d}\textstyle\sum_{i=1}^{d} S_i(T) - K\Big)^+. \tag{4}$$

BSDE. Let $X = (S_1, \dots, S_d, v_1, \dots, v_d)$ and let $u(t,x)$ be the price function, which solves $\partial_t u + \mathcal{L}u - ru = 0$, $u(T,\cdot) = g$, with $\mathcal{L}$ the generator of (2)–(3). Writing $\overline{W} = (W_1, \dots, W_d, B_1, \dots, B_d)$, Itô's formula applied to $Y_t = u(t, X_t)$ gives the linear BSDE

$$dY_t = rY_t\,dt + Z_t \cdot d\overline{W}_t, \qquad Y_T = g(S(T)), \tag{5}$$

with $Z_i^S = \sqrt{v_i}\,S_i\,\partial_{S_i}u$ and $Z_i^v = \sigma_v\sqrt{v_i}\,\partial_{v_i}u$. The price is $Y_0 = u(0, X_0)$, and the delta of asset i is recovered as $\Delta_i = Z_i^S(0)/(\sqrt{v_0}\,S_0)$. (The representation in the correlated basis $\overline{W}$ is unique because the covariance of $\overline{W}$ is non-singular for $|\rho| < 1$.)

3 Method

3.1 Time discretization of the forward process

On a uniform grid $t_n = n\Delta t$, $\Delta t = T/N$, write $v^+ = \max(v, 0)$ and let $\Delta W_{i,n}, \Delta B_{i,n}$ be the Brownian increments. We compare two schemes.

PT (partial truncation). Euler–Maruyama for S_i and for v_i, with the negative part removed only inside the square root:

$$S_{i,n+1} = S_{i,n} + rS_{i,n}\Delta t + \sqrt{v_{i,n}^+}\,S_{i,n}\Delta W_{i,n},$$

$$v_{i,n+1} = v_{i,n} + \kappa(\theta - v_{i,n})\Delta t + \sigma_v\sqrt{v_{i,n}^+}\,\Delta B_{i,n}.$$

FT (full truncation with log-Euler price step) [22]:

$$S_{i,n+1} = S_{i,n}\exp\Big(\big(r - \tfrac{1}{2}v_{i,n}^+\big)\Delta t + \sqrt{v_{i,n}^+}\,\Delta W_{i,n}\Big),$$

$$v_{i,n+1} = v_{i,n} + \kappa(\theta - v_{i,n}^+)\Delta t + \sigma_v\sqrt{v_{i,n}^+}\,\Delta B_{i,n}.$$

Both schemes are driven by Brownian increments only, so either can be used inside the Deep BSDE method without modification. (Exact or QE sampling of the CIR transition [5, 7] cannot be used directly, since the BSDE recursion needs the Brownian increments that drive the variance.)

3.2 Deep BSDE solver

Following [16], the BSDE (5) is discretized forward in time:

$$Y_{n+1} = (1 + r\Delta t)\, Y_n + Z_n \cdot \Delta \overline{W}_n, \qquad n = 0, \dots, N-1. \tag{6}$$

The initial value $Y_0 \in \mathbb{R}$ and the initial gradient $Z_0 \in \mathbb{R}^{2d}$ are trainable parameters (since X_0 is deterministic). For $n \geq 1$, $Z_n = s\,\phi_n(\tilde{X}_n; \vartheta_n)$, where $\phi_n : \mathbb{R}^{2d} \to \mathbb{R}^{2d}$ is a separate fully connected network for each time step, with two hidden layers of width $d+10$ and ReLU activations (the width convention of [16]). The inputs are normalized as $\tilde{S}_i = (S_i/S_0 - 1)/\sqrt{\theta}$ and $\tilde{v}_i = (v_i - \theta)/\theta$, and the outputs are scaled by $s = S_0\sqrt{\theta}$. All parameters are trained jointly to minimize the mean squared terminal mismatch

$$\mathcal{L} = \frac{1}{M} \sum_{m=1}^{M} \big(Y_N^{(m)} - g(S_N^{(m)})\big)^2 \tag{7}$$

over fresh batches of $M = 256$ simulated paths, using Adam [21] for 4,000 iterations with $N = 40$ time steps. Learning rates are $3 \cdot 10^{-3}$ for the networks, $5 \cdot 10^{-2}$ for Y_0 and 10^{-2} for Z_0, multiplied by 0.1 after 50% and again after 80% of the iterations. Y_0 is initialized with an uninformative random value (no Monte Carlo warm start; see Section 4.5 for the warm-start ablation). The architecture and iteration budget follow the same rule for every d, so that changes with d are not confounded by hand-tuned capacity.

For a linear BSDE, the minimizer of (7) over Y_0 and adapted Z satisfies $Y_0 = (1+r\Delta t)^{-N}\, \mathbb{E}[g(S_N)]$. The best price the solver can achieve is therefore the discounted expectation under the *discretized* dynamics. (The difference between $(1 + r\Delta t)^{-N}$ and e^{-rT} is below 10^{-5} in relative terms here.)

3.3 Reference prices and error decomposition

True prices. For $d = 1$ we use the semi-closed-form Heston price, computed by Fourier inversion of the characteristic function in the numerically stable form of [2]. For $d > 1$ (with $\rho_S = 0$) we use the QE scheme of Andersen [5] with martingale correction and 50 time steps ($\Delta t = 0.01$), with 10^7–$4 \cdot 10^7$ paths and the basket mean as a control variate. Its expectation $S_0 e^{rT}$ is exact under the martingale-corrected scheme. For $\rho_S > 0$, where QE does not directly apply, we use the FT scheme with $N = 400$. At $d = 1$ this fine-grid reference differs from the exact price by at most 0.05%, which bounds its accuracy (Section 5).

Error decomposition. Let u be the true price, u_{sch}^N the price under the discretized dynamics used for training (scheme sch with N steps), and $\hat{Y}_0$ the trained Deep BSDE price. We estimate u_{sch}^N by an independent large Monte Carlo run of the same discretized dynamics (10^7–$4 \cdot 10^7$ paths in the main experiment and at least $4 \cdot 10^6$ elsewhere, with a control variate) and split the relative error as

$$\underbrace{\frac{\hat{Y}_0 - u}{u}}_{\text{total error}} = \underbrace{\frac{u_{\text{sch}}^N - u}{u}}_{\text{discretization bias}} + \underbrace{\frac{\hat{Y}_0 - u_{\text{sch}}^N}{u}}_{\text{solver error}}. \tag{8}$$

The discretization bias depends only on the forward scheme and therefore cannot be reduced by changing the network or its training.

Hedge quality. We compare the learned initial delta $\hat{\Delta} = \frac{1}{d}\sum_i Z_i^S(0)/(\sqrt{v_0}S_0)$ with the pathwise Monte Carlo delta $e^{-rT}\mathbb{E}\big[\mathbf{1}\{\bar{S}_T > K\}\, \frac{1}{d}\sum_i S_i(T)/(dS_0)\big]$ (per asset, averaged over assets by

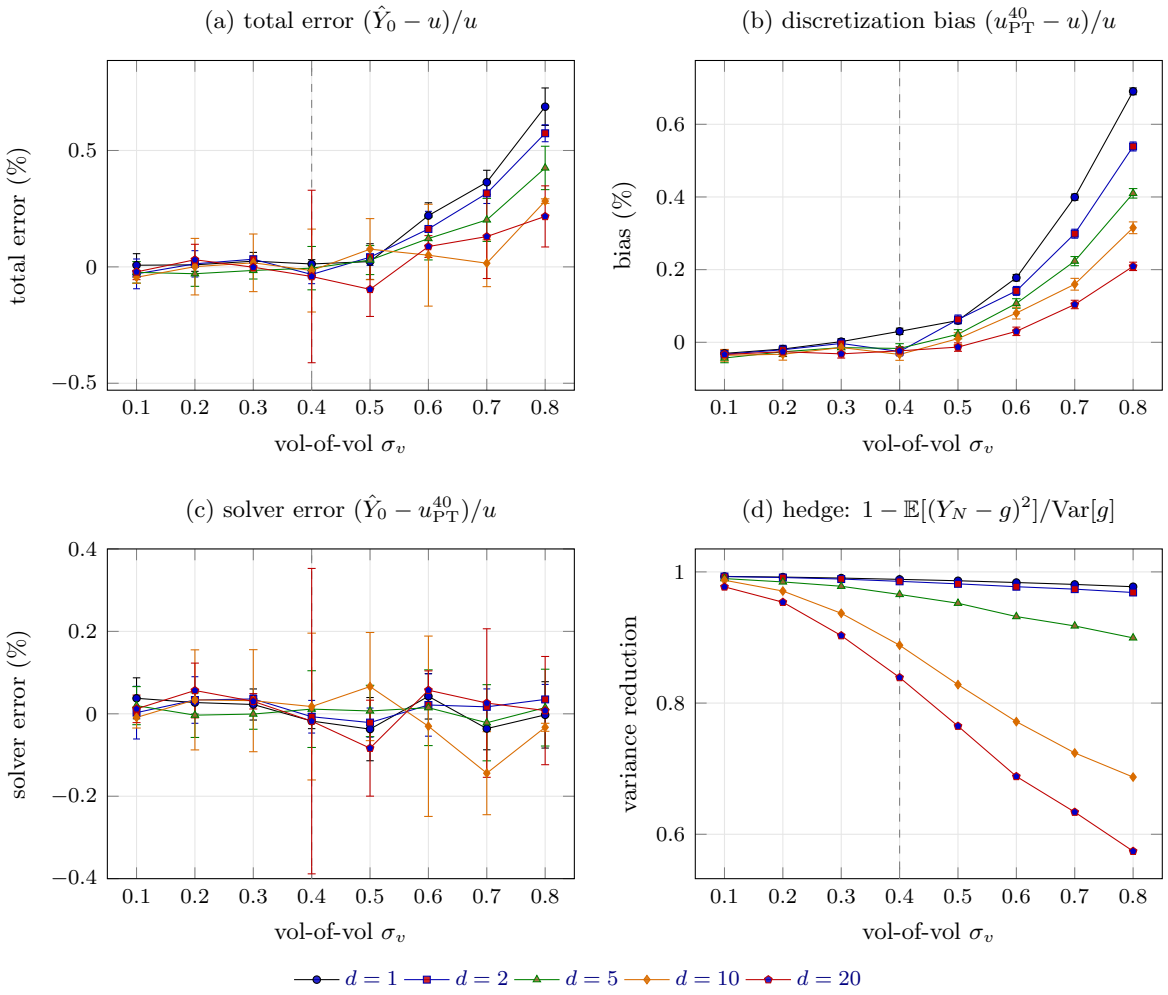


Figure 1: Main experiment (PT scheme, $N = 40$, $\kappa = 2$, independent assets). Error bars are ± 1 standard deviation over seeds (5 seeds for $d \leq 5$, 3 for $d \geq 10$) in (a) and (c), and ± 1 Monte Carlo standard error in (b). The dashed line marks the Feller threshold $\sigma_v^* = 0.4$. Almost all of the total error (a) is discretization bias (b), which can be measured without any neural network. The solver error (c) stays small and shows no systematic trend with σ_v. Hedge quality (d) degrades with both σ_v and d.

symmetry), or with the analytic delta for $d = 1$. Because Y_N is the terminal value of the self-financing strategy defined by (Y_0, Z), the loss (7) equals the mean squared replication error. We report it on 8,192 fresh validation paths as a variance reduction $1 - \mathbb{E}[(Y_N - g)^2]/\mathrm{Var}[g]$ relative to not hedging, and as a relative root-mean-square replication error $\sqrt{\mathbb{E}[(Y_N - g)^2]}/\hat{Y}_0$.

Implementation. We implemented the method twice and cross-checked the two implementations (Section 4.1). The first is in JavaScript (Node.js), with hand-coded backpropagation verified against central finite differences (agreement to at least five significant digits on all checked parameters). The second is in PyTorch and trains many independent solvers in parallel on one NVIDIA T4 GPU. All reported results use the PyTorch implementation, single-precision training and double-precision Monte Carlo accumulation. Code, configurations, seeds and the raw results of every run are publicly available (see Code and data availability).

d	0.1	0.2	0.3	0.4	**0.5**	**0.6**	**0.7**	**0.8**
				vol-of-vol σ_v (bold: Feller violated, $\sigma_v > 0.4$)				
(a) Total error $(\hat{Y}_0 - u)/u$, %: mean over seeds (standard deviation)								
1	0.01 (0.05)	0.01 (0.03)	0.02 (0.04)	0.01 (0.02)	0.02 (0.08)	0.22 (0.06)	0.36 (0.05)	0.69 (0.08)
2	−0.03 (0.06)	0.01 (0.06)	0.03 (0.01)	−0.03 (0.04)	0.04 (0.04)	0.16 (0.08)	0.32 (0.04)	0.57 (0.04)
5	−0.02 (0.05)	−0.03 (0.05)	−0.02 (0.04)	−0.01 (0.09)	0.03 (0.06)	0.12 (0.09)	0.20 (0.09)	0.43 (0.09)
10	−0.04 (0.03)	0.00 (0.12)	0.02 (0.12)	−0.02 (0.18)	0.08 (0.13)	0.05 (0.22)	0.02 (0.10)	0.28 (0.01)
20	−0.02 (0.03)	0.03 (0.07)	0.00 (0.02)	−0.04 (0.37)	−0.10 (0.12)	0.09 (0.05)	0.13 (0.18)	0.22 (0.13)
(b) Discretization bias $(u_{\mathrm{PT}}^{40} - u)/u$, %, measured by Monte Carlo without any network (MC std. error)								
1	−0.03 (0.01)	−0.02 (0.01)	0.00 (0.01)	0.03 (0.01)	0.06 (0.01)	0.18 (0.01)	0.40 (0.01)	0.69 (0.01)
2	−0.03 (0.01)	−0.02 (0.01)	0.00 (0.01)	−0.03 (0.01)	0.06 (0.01)	0.14 (0.01)	0.30 (0.01)	0.54 (0.01)
5	−0.04 (0.01)	−0.03 (0.01)	−0.01 (0.01)	−0.02 (0.01)	0.02 (0.01)	0.11 (0.01)	0.22 (0.01)	0.41 (0.01)
10	−0.04 (0.02)	−0.03 (0.02)	−0.01 (0.02)	−0.03 (0.02)	0.01 (0.02)	0.08 (0.02)	0.16 (0.02)	0.32 (0.02)
20	−0.03 (0.01)	−0.03 (0.01)	−0.03 (0.01)	−0.02 (0.01)	−0.01 (0.01)	0.03 (0.01)	0.10 (0.01)	0.21 (0.01)
(c) Solver error $(\hat{Y}_0 - u_{\mathrm{PT}}^{40})/u$, %: mean over seeds								
1	0.04	0.03	0.02	−0.02	−0.04	0.04	−0.04	0.00
2	0.00	0.03	0.04	−0.01	−0.02	0.02	0.02	0.03
5	0.02	0.00	0.00	0.01	0.01	0.01	−0.02	0.01
10	−0.01	0.03	0.03	0.02	0.07	−0.03	−0.14	−0.03
20	0.01	0.06	0.03	−0.02	−0.08	0.06	0.03	0.01
(d) Reference price u (exact for $d = 1$; QE Monte Carlo for $d > 1$)								
1	6.900	6.882	6.836	6.762	6.665	6.551	6.428	6.301
2	5.300	5.284	5.260	5.230	5.192	5.151	5.105	5.054
5	3.919	3.909	3.896	3.883	3.867	3.851	3.836	3.819
10	3.265	3.259	3.250	3.242	3.233	3.224	3.215	3.206
20	2.851	2.846	2.842	2.837	2.832	2.827	2.822	2.817

Table 1: Main experiment: Deep BSDE with the PT scheme and $N = 40$ time steps, $\kappa = 2$, independent assets. Five seeds for $d \leq 5$ and three for $d \geq 10$. By construction (a) = (b) + (c), up to the Monte Carlo error of the references.

4 Results

4.1 Validation of the reference prices and the solver

Over the 57 combinations (κ, σ_v) with $\kappa \in \{1, 2, 4\}$ and $\sigma_v \in \{0.10, 0.15, \ldots, 1.00\}$ at $d = 1$, the QE Monte Carlo prices agree with the semi-closed-form price within their statistical error. The largest deviation is 0.088% of the price (2.7 standard errors over 57 comparisons), and the pathwise deltas agree with the analytic delta to within 0.0007 in absolute terms. We implemented the full pipeline twice: in single-threaded JavaScript, and in PyTorch on a GPU, where independent runs are trained in parallel. The two implementations agree to within Monte Carlo error on all quantities we compared (for example, Euler prices at $d = 1$, $\sigma_v = 0.8$ of 6.3438 ± 0.0029 and 6.3472 ± 0.0020; QE prices at $d = 20$, $\sigma_v = 0.4$ of 2.8361 ± 0.0007 and 2.8376 ± 0.0007; and Deep BSDE prices at $d = 20$, $\sigma_v = 0.1$ in the range 2.849–2.853 in both). All results below come from the GPU implementation.

4.2 The error landscape and its decomposition

Table 1 and Figure 1 summarize the main experiment. In the regime where the Feller condition holds ($\sigma_v \leq 0.4$), the total error is at most 0.04% in absolute value at every dimension. It is

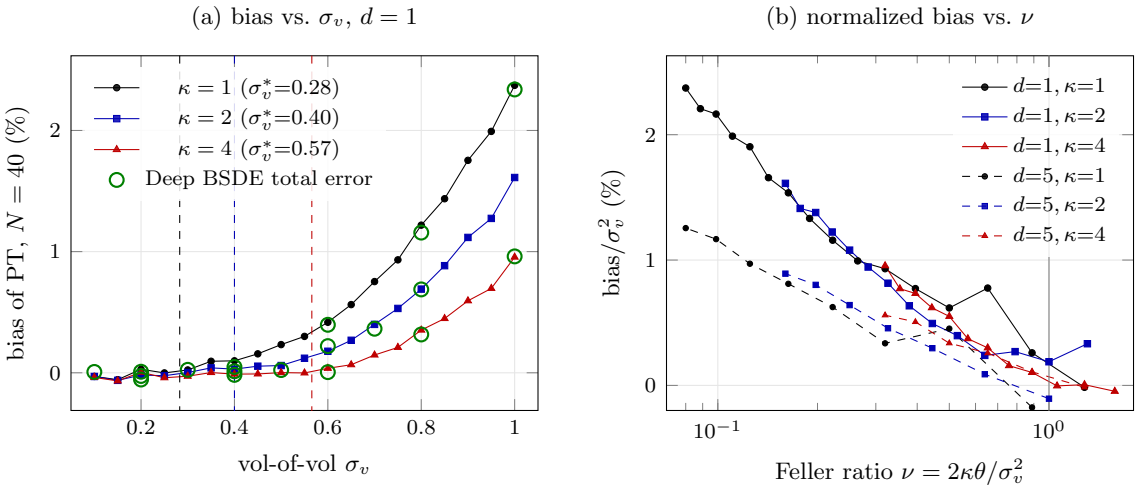


Figure 2: Moving the Feller threshold. (a) Bias of the PT scheme with $N = 40$ (lines; at least $8 \cdot 10^6$ paths per point, against the exact Heston price) and the total error of the trained Deep BSDE solver (circles; mean over 3 to 5 seeds) for $d = 1$ and $\kappa \in \{1, 2, 4\}$. Dashed vertical lines mark each $\sigma_v^* = \sqrt{2\kappa\theta}$. (b) The same biases divided by σ_v^2 and plotted against the Feller ratio ν, for $d = 1$ (solid) and $d = 5$ (dashed). Only points with $\nu \leq 1.6$ are shown; for larger ν the bias is within Monte Carlo noise and dividing by a small σ_v^2 only amplifies that noise. For each d, the curves for different κ approximately collapse onto a single curve.

statistically indistinguishable from zero at most grid points, given the seed-to-seed spread. Past the threshold the error grows with σ_v and is always positive, reaching 0.69% at $(d, \sigma_v) = (1, 0.8)$ and 0.22% at $(20, 0.8)$.

The decomposition (8) shows where this error comes from. Panel (b) of Table 1 gives the bias of the training dynamics, measured by Monte Carlo alone, without any network. It reproduces the total error almost exactly. For instance, at $(1, 0.8)$ the bias is 0.69% against a total error of 0.69%, and at $(20, 0.8)$ it is 0.21% against 0.22%. The solver error (panel (c)) is small at every grid point, with mean absolute value 0.029% over the 40 grid points and a maximum of 0.14%. It also shows no trend with σ_v: its average over the Feller-violated half of the grid (0.036%) is no larger than over the Feller-satisfied half (0.021%). The trained networks therefore converge to the price of the discretized model equally well on both sides of the Feller boundary; the loss of accuracy past the boundary is a property of the discretized model. Seed-to-seed variability does grow with dimension, from at most 0.08% at $d \leq 2$ to 0.1–0.37% at $d \in \{10, 20\}$. In high dimension, single runs are therefore noisier than the bias at moderate σ_v, and averaging a few seeds is advisable.

4.3 Moving the Feller threshold

If the Feller condition governs the onset of the error, the onset should shift with the threshold. We changed the mean-reversion speed to $\kappa \in \{1, 2, 4\}$ (thresholds $\sigma_v^* = 0.28$, 0.40, 0.57) and measured the bias of the PT scheme on a fine σ_v grid (Figure 2a). We also trained Deep BSDE solvers at $d = 1$ for $\kappa \in \{1, 4\}$. Their total errors (circles) again sit on the bias curves; the solver error has mean absolute value 0.036% over these runs.

Three observations follow. First, the onset moves with the threshold. Taking "onset" to mean the smallest σ_v at which the bias exceeds three Monte Carlo standard errors, it occurs at $\sigma_v = 0.35$, 0.50 and 0.65 for $\kappa = 1, 2, 4$, corresponding to Feller ratios $\nu = 0.65$, 0.64 and 0.76. The degradation therefore starts somewhat beyond the Feller boundary (ν between roughly 0.6 and 0.8, not at $\nu = 1$), and it sets in gradually rather than as a sharp break. Second, at a fixed σ_v the bias is much smaller

Variant	$d = 2$		$d = 10$		$d = 10,\ \sigma_v$=0.7
	$\sigma_v = 0.3$	$\sigma_v = 0.7$	$\sigma_v = 0.3$	$\sigma_v = 0.7$	var. red.
Baseline (N=40, width d+10, 2 layers, 4k it., M=256)	0.03 (0.01)	0.32 (0.04)	0.02 (0.12)	0.02 (0.10)	0.724
Wider (width 4(d+10))	−0.02 (0.06)	0.32 (0.08)	−0.05 (0.00)	0.15 (0.20)	0.774
Deeper (3 hidden layers)	0.01 (0.07)	0.32 (0.09)	−0.03 (0.09)	0.22 (0.20)	0.697
Longer training (10k iterations)	−0.03 (0.07)	0.25 (0.07)	−0.10 (0.22)	0.05 (0.09)	0.812
Larger batch ($M = 1024$)	−0.04 (0.00)	0.28 (0.02)	−0.02 (0.06)	0.14 (0.07)	0.858
Monte Carlo warm start of Y_0	−0.01 (0.08)	0.26 (0.13)	−0.04 (0.03)	0.27 (0.14)	0.721
More time steps ($N = 80$)	−0.07 (0.05)	0.11 (0.03)	0.05 (0.02)	0.09 (0.12)	0.675
FT scheme instead of PT	0.01 (0.05)	0.28 (0.04)	0.01 (0.07)	0.01 (0.08)	0.717

Table 2: Ablations. Entries are the total pricing error in % (mean over 3 seeds, with the standard deviation in parentheses; the baseline row uses the 5 or 3 seeds of the main experiment). The last column is the hedge variance reduction. Only changing the time discretization moves the pricing error systematically. Network capacity, training length and batch size improve the hedge but not the price.

when κ is larger, because the Feller ratio is larger. At $\sigma_v = 0.8$, for example, the bias is 1.22%, 0.69% and 0.35% for $\kappa = 1, 2, 4$. Third, plotting the bias divided by σ_v^2 against ν (Figure 2b) approximately collapses the three curves for each dimension. At $\nu \approx 0.16$ the normalized bias at $d = 1$ is 1.54 ($\kappa = 1$) and 1.61 ($\kappa = 2$). At $\nu \approx 0.32$ it is 0.93 ($\kappa = 1$) and 0.95 ($\kappa = 4$). The Feller ratio is therefore the natural coordinate for the onset, but σ_v separately sets the size of the bias. We do not have a theoretical derivation of this empirical scaling and report it as an observation.

4.4 Time steps and discretization scheme

Since the bias belongs to the time-discretized forward process, it should decrease as the grid is refined. Figure **??** confirms this. For the PT scheme at $d = 1$ and $\sigma_v = 0.7$, the bias falls from 2.38% at $N = 10$ to 0.95%, 0.40%, 0.14% and 0.05% at $N = 20, 40, 80, 160$. That is a factor of about 2.4–3 per doubling, consistent with (slightly better than) first-order weak convergence. At $d = 5$ it falls from 1.35% to 0.01%. The trained Deep BSDE solvers follow these curves closely. Over all 36 combinations of $d \in \{1, 5\}$, $\sigma_v \in \{0.3, 0.5, 0.7\}$, scheme and N that we trained, the solver error is at most 0.07% in absolute value, while the total error ranges from −0.05% to 2.46%. Refining the time grid is therefore an effective and predictable way to reduce the pricing error. Its cost is linear in N, since both the simulation and the number of sub-networks scale with N.

We also replaced the PT scheme by full truncation with a log-Euler price step (FT), which Lord et al. [22] found to be the least biased of the simple Euler fixes in their Heston Monte Carlo study. In our setting FT offers no improvement. Its bias is essentially identical to that of PT at $\sigma_v = 0.7$ (e.g. 0.386% vs. 0.399% at $d = 1$, $N = 40$) and larger at $\sigma_v \leq 0.5$ (e.g. 0.147% vs. 0.060% at $d = 1$, $\sigma_v = 0.5$, $N = 40$). The choice between these two truncation variants is thus second-order compared with the choice of N.

4.5 Ablations: what does and does not reduce the error

Table 2 varies one design choice at a time at $d \in \{2, 10\}$ and $\sigma_v \in \{0.3, 0.7\}$. At $d = 2$ and $\sigma_v = 0.7$, the bias (0.32%) is large compared with the seed-to-seed spread (0.04%). There, only doubling the number of time steps reduces the error substantially, to 0.11%. Wider or deeper networks, 2.5× longer training, a 4× larger batch, a Monte Carlo warm start of Y_0 and the FT scheme all leave it between 0.25% and 0.32%. At $d = 10$ the seed-to-seed standard deviation (up to 0.22%) is as large

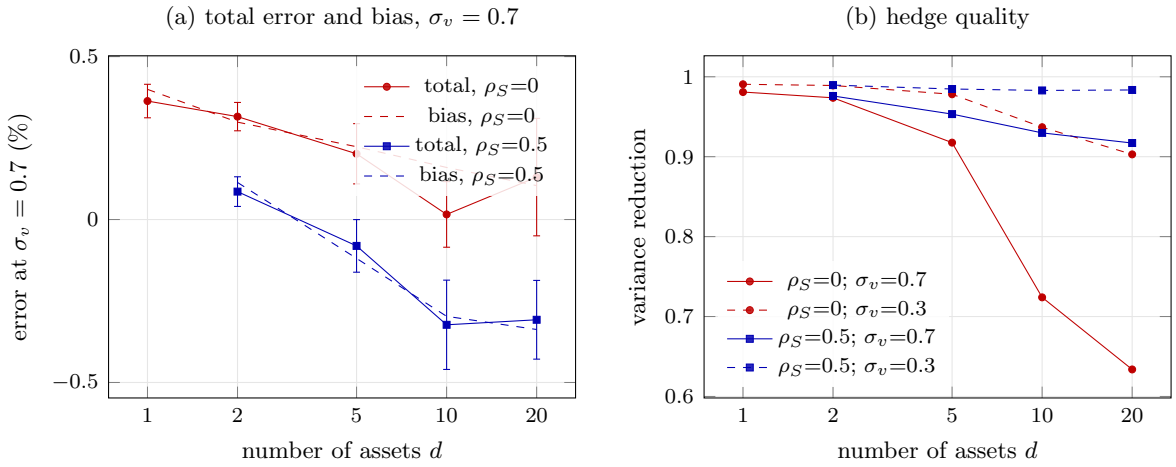


Figure 3: Effect of the number of assets and of inter-asset correlation ρ_S. (a) Total error of the Deep BSDE price (solid; ± 1 s.d. over seeds) and discretization bias (dashed) at $\sigma_v = 0.7$. (b) Hedge variance reduction at $\sigma_v \in \{0.3, 0.7\}$. For $\rho_S = 0.5$ the reference is the FT scheme with $N = 400$, whose own bias is at most 0.05% at $d = 1$ (Section 3.3).

as the bias itself (0.16% at $\sigma_v = 0.7$). The price ablations cannot be resolved at this level, and no variant gives a consistent improvement at both values of σ_v. Warm-starting Y_0 with a 5,000-path Monte Carlo estimate does not improve accuracy at either dimension. The accuracy of our prices therefore does not depend on initializing Y_0 near the answer, since all main runs start from an uninformative value. The hedge responds differently. At $d = 10$, $\sigma_v = 0.7$ its variance reduction rises from 0.72 (baseline) to 0.77 with wider networks, 0.81 with longer training and 0.86 with a larger batch. It falls slightly with a deeper network (0.70) and with more time steps at a fixed iteration budget (0.68).

4.6 Dimension, correlation and the basket effect

At a fixed Feller-violating σ_v, the total error decreases with the number of assets (Table 1a; Figure 3a). The decomposition attributes this decrease to a smaller bias of the discretized dynamics rather than to a more accurate network. At $\sigma_v = 0.8$ the bias falls from 0.69% at $d = 1$ to 0.21% at $d = 20$. For independent assets the basket averages d independent variance paths, so the payoff depends on the variance processes mainly through their average, and errors made path by path near $v = 0$ partly cancel. The vol-of-vol sensitivity of the true price itself shrinks in the same way: the reference price varies by 8.7% across $\sigma_v \in [0.1, 0.8]$ at $d = 1$ but only by 1.2% at $d = 20$.

With equicorrelated assets ($\rho_S = 0.5$; Figure 3a) the behavior is different. The network still converges to the price of the discretized dynamics: the solver error lies between -0.03% and $+0.04\%$ at all eight (d, σ_v) combinations. The discretization bias, however, no longer fades with d. At $\sigma_v = 0.7$ it is $+0.11\%$ at $d = 2$ but -0.12%, -0.30% and -0.34% at $d = 5, 10, 20$, so that the Euler scheme underprices. To check that this is a property of the discretization and not of the reference, we recomputed the $d = 10$, $\sigma_v = 0.7$ price with $N = 40$, 160 and 640 steps under both schemes. Both converge from below to the same value (4.898, 4.903, 4.914 for PT; 4.893, 4.902, 4.914 for FT; reference 4.913), and the $d = 5$ case behaves the same way. The convergence between $N = 40$ and $N = 160$ is noticeably slower than $1/N$. We do not yet have an explanation for the sign change or the slower convergence. A plausible factor is the dependence across assets that ρ_S induces between the variance processes (correlation $\rho^2 \rho_S \approx 0.25$), which prevents the variance errors from

d	0.1	0.2	0.3	0.4	**0.5**	**0.6**	**0.7**	**0.8**
(a) Initial delta: mean absolute relative error vs. reference delta, %								
1	0.35	0.24	0.30	0.11	0.26	0.20	0.31	0.37
2	0.29	0.26	0.15	0.19	0.21	0.22	0.35	0.36
5	0.27	0.24	0.29	0.28	0.58	0.46	0.19	0.13
10	0.26	0.44	0.45	0.79	1.22	0.61	0.89	0.83
20	0.43	0.42	0.30	1.10	1.03	0.90	1.00	1.63
(b) Replication: variance reduction $1 - \mathbb{E}[(Y_N - g)^2]/\mathrm{Var}[g]$ *on fresh paths*								
1	0.993	0.992	0.991	0.989	0.987	0.984	0.981	0.977
2	0.993	0.991	0.989	0.986	0.982	0.977	0.974	0.969
5	0.990	0.985	0.978	0.966	0.952	0.932	0.918	0.899
10	0.987	0.971	0.937	0.888	0.828	0.772	0.724	0.687
20	0.977	0.954	0.903	0.839	0.765	0.688	0.634	0.574
(c) Relative RMS replication error $\sqrt{\mathbb{E}[(Y_N - g)^2]}/\hat{Y}_0$, *%*								
1	11.4	11.6	12.1	12.7	13.2	14.2	15.1	16.1
2	11.0	11.6	12.6	14.0	15.3	16.5	17.5	18.7
5	11.9	14.3	16.7	20.3	23.4	27.7	29.9	32.5
10	11.9	17.8	25.8	33.7	41.1	47.0	51.1	53.6
20	13.9	19.7	28.4	36.0	42.5	48.4	52.1	55.4

Table 3: Hedge quality in the main experiment, averaged over seeds. The reference delta is analytic for $d = 1$ and a pathwise Monte Carlo estimate (QE scheme) for $d > 1$. Replication statistics use 8,192 fresh paths per run.

averaging out. Hence the magnitude, and even the sign, of the discretization bias depends on the dependence structure and should be measured for the model at hand rather than assumed to be small.

The decrease of the error with d is therefore not evidence that the method escapes the curse of dimensionality in general. It is a property of an averaging payoff on independent assets, and, as the correlated case shows, it is not robust to dependence between the assets.

4.7 Hedging

The loss (7) is the mean squared error of the replication strategy defined by the learned Z, so it measures hedge quality directly. Table 3 reports three hedge metrics. The initial delta is accurate everywhere: its error is at most 1.7% at every grid point and below 0.6% for $d \leq 5$. The pathwise hedge, in contrast, degrades markedly with both σ_v and d. The variance reduction of the replication strategy falls from 0.993 at $(1, 0.1)$ to 0.57 at $(20, 0.8)$. Equivalently, the relative RMS replication error rises from about 11% at $(1, 0.1)$ to 55% at $(20, 0.8)$. This deterioration does not appear in the price, because errors in Z enter Y_N only through stochastic integrals with zero mean. They add variance to the terminal mismatch but do not shift the optimal Y_0. The ablations show that wider networks, longer training and larger batches improve the hedge but not the price (last column of Table 2). The hedge, unlike the price, is limited by approximation and optimization. In practice, a Deep BSDE price can be accurate while the learned hedging strategy is still far from optimal, so the two should be validated separately.

4.8 Computational cost

With the width rule $d + 10$, the number of trainable parameters grows from 7,374 at $d = 1$ to 132,641 at $d = 20$ (for $N = 40$). On the GPU, a single 4,000-iteration run takes 72–76 seconds for every $d \in \{1, \ldots, 20\}$. A batch of up to 40 independent runs trained in parallel takes 73–83 seconds. At these problem sizes the cost is dominated by the sequential 40-step recursion and kernel-launch overhead, not by arithmetic. On a single CPU thread (JavaScript implementation, Intel i9-9980HK) the same run took about 43, 117, 210 and 510 seconds for $d = 1, 5, 10, 20$, which grows roughly in proportion to the parameter count. For comparison, all reference prices of the main experiment (QE and PT schemes, 40 configurations, $2 \cdot 10^6$–$8 \cdot 10^6$ paths each) took 140 seconds on the same GPU.

5 Discussion

Implications for practice. For European-style multi-asset Heston problems, our results suggest a simple workflow. (i) Before training, estimate the bias of the forward scheme by plain Monte Carlo on the same time grid. This costs seconds and bounds the accuracy any network can reach. (ii) Choose N so that this bias is below the target tolerance. Because the bias decays roughly like $1/N$, halving it costs about twice the training time. (iii) Validate prices and hedges separately. With independent assets, the Feller ratio ν of each variance process indicates when (ii) matters: for $\nu \gtrsim 1$ the bias of a 40-step grid was below about 0.05% in all our tests, whereas for $\nu \approx 0.25$ it was 0.3–0.7% at $d = 1$. Market calibrations frequently have $\nu < 1$ [2, 5], so the time grid deserves explicit attention when Deep BSDE solvers are used with realistic Heston parameters.

Monte Carlo versus Deep BSDE. For the European basket prices studied here, Monte Carlo with a control variate is far cheaper than Deep BSDE training: all reference prices for one dimension (eight values of σ_v, $2 \cdot 10^6$–$8 \cdot 10^6$ paths each) took under a minute on the GPU, roughly the time needed to train one batch of solvers. We therefore do not claim a speed advantage for Deep BSDE on this problem. Its value lies in producing a hedging strategy along with the price, and in extending to nonlinear pricing problems (e.g. funding or counterparty adjustments, or American features via reflected BSDEs [8, 23]) where plain Monte Carlo is not directly applicable. Our findings carry over to those settings in one important respect: the forward discretization bias is inherited by any solver trained on the discretized dynamics.

Limitations. (i) We study a single payoff (an at-the-money arithmetic basket call), a single maturity ($T = 0.5$) and one family of parameters; out-of-the-money options and longer maturities may weigh the variance near zero differently. (ii) Each asset has its own variance process and the assets are independent or equicorrelated. Models with a common variance factor would not benefit from the averaging effect of Section 4.6. (iii) We consider $d \leq 20$ and only the original Deep BSDE architecture; newer variants [6, 19] may behave differently, although the discretization bias is solver-independent. (iv) The scaling of Figure 2b is empirical. A theoretical explanation, possibly building on the error analysis of [20] for Hölder-continuous diffusion coefficients and the CIR discretization results of [3, 18], is left for future work. (v) For correlated assets the reference uses a fine-grid Euler scheme rather than an exact or QE scheme. Its residual bias of at most 0.05% (at $d = 1$) limits the precision of that comparison.

Future work. Positivity-preserving implicit schemes for the CIR process [3, 4] are driven by Brownian increments and could be used inside the BSDE recursion. They are the natural next

candidates to reduce the bias at fixed N. Further directions are adaptive time grids that are finer where the variance is small, and an analysis of the observed $\sigma_v^2 F(\nu)$ scaling.

6 Conclusion

Across 282 trained solvers spanning $d \in \{1, \ldots, 20\}$ and $\sigma_v \in [0.1, 0.8]$, the Deep BSDE method priced multi-asset Heston basket options to within the bias of the discretized forward dynamics it was trained on. The neural solver error was small (typically below 0.1%) and did not grow past the Feller boundary. The accuracy loss in the Feller-violating regime came almost entirely from the Euler discretization of the variance process. This bias begins once the Feller ratio falls below roughly 0.7–0.8, scales approximately like $\sigma_v^2 F(\nu)$, decays at first order in the time step, and shrinks with the number of independent assets in the basket, although not when the assets are correlated. Network capacity, training length, batch size and warm starts did not affect the price, although they did improve the hedge. The hedge quality deteriorated substantially with d and σ_v even where prices were accurate. Error budgets for neural PDE solvers in finance should therefore report the discretization bias and the solver error separately, and should validate hedges independently of prices.

Code and data availability

All code (PyTorch and JavaScript implementations, experiment drivers and analysis scripts) and the raw per-run results are available at `https://github.com/lnxv/deep-bsde-heston`.

Acknowledgments

We thank Dr. Jie Zhong (California State University, Los Angeles) for guidance on the research direction and feedback on the manuscript. AI-based tools were used to assist with code development, running experiments and editing the manuscript; the authors reviewed all code, results and text and take full responsibility for the content.

References

[1] B. A. Aaslund, J. Berge, Y. Ni, and R. Pimentel. Neural network-based pricing of high-dimensional Bermudan basket options under stochastic volatility. *Networks and Heterogeneous Media*, 20(3):759–781, 2025. doi: 10.3934/nhm.2025032.

[2] H. Albrecher, P. Mayer, W. Schoutens, and J. Tistaert. The little Heston trap. *Wilmott Magazine*, pages 83–92, Jan. 2007.

[3] A. Alfonsi. On the discretization schemes for the CIR (and Bessel squared) processes. *Monte Carlo Methods and Applications*, 11(4), 2005. doi: 10.1515/156939605777438569.

[4] A. Alfonsi. High order discretization schemes for the CIR process: Application to affine term structure and Heston models. *Mathematics of Computation*, 79(269):209–237, 2010. doi: 10.1090/s0025-5718-09-02252-2.

[5] L. B. G. Andersen. Simple and efficient simulation of the Heston stochastic volatility model. *The Journal of Computational Finance*, 11(3):1–42, 2008. doi: 10.21314/jcf.2008.189.

[6] C. Beck, W. E, and A. Jentzen. Machine learning approximation algorithms for high-dimensional fully nonlinear partial differential equations and second-order backward stochastic differential equations. *Journal of Nonlinear Science*, 29(4):1563–1619, 2019. doi: 10.1007/s00332-018-9525-3.

[7] M. Broadie and Ö. Kaya. Exact simulation of stochastic volatility and other affine jump diffusion processes. *Operations Research*, 54(2):217–231, 2006. doi: 10.1287/opre.1050.0247.

[8] Y. Chen and J. W. L. Wan. Deep neural network framework based on backward stochastic differential equations for pricing and hedging American options in high dimensions. *Quantitative Finance*, 21(1):45–67, 2021. doi: 10.1080/14697688.2020.1788219.

[9] J. C. Cox, J. E. Ingersoll, and S. A. Ross. A theory of the term structure of interest rates. *Econometrica*, 53(2):385–407, 1985. doi: 10.2307/1911242.

[10] W. E, J. Han, and A. Jentzen. Deep learning-based numerical methods for high-dimensional parabolic partial differential equations and backward stochastic differential equations. *Communications in Mathematics and Statistics*, 5(4):349–380, 2017. doi: 10.1007/s40304-017-0117-6.

[11] N. El Karoui, S. Peng, and M. C. Quenez. Backward stochastic differential equations in finance. *Mathematical Finance*, 7(1):1–71, 1997. doi: 10.1111/1467-9965.00022.

[12] W. Feller. Two singular diffusion problems. *The Annals of Mathematics*, 54(1):173–182, 1951. doi: 10.2307/1969318.

[13] M. Germain, H. Pham, and X. Warin. Neural networks-based algorithms for stochastic control and PDEs in finance, 2021. arXiv:2101.08068.

[14] A. Gnoatto, S. Lavagnini, and A. Picarelli. Deep quadratic hedging. *Mathematics of Operations Research*, 50(4):2972–3009, 2025. doi: 10.1287/moor.2023.0213.

[15] J. Han and J. Long. Convergence of the deep BSDE method for coupled FBSDEs. *Probability, Uncertainty and Quantitative Risk*, 5(1):5, 2020. doi: 10.1186/s41546-020-00047-w.

[16] J. Han, A. Jentzen, and W. E. Solving high-dimensional partial differential equations using deep learning. *Proceedings of the National Academy of Sciences*, 115(34):8505–8510, 2018. doi: 10.1073/pnas.1718942115.

[17] S. L. Heston. A closed-form solution for options with stochastic volatility with applications to bond and currency options. *The Review of Financial Studies*, 6(2):327–343, 1993. doi: 10.1093/rfs/6.2.327.

[18] D. J. Higham and X. Mao. Convergence of monte carlo simulations involving the mean-reverting square root process. *The Journal of Computational Finance*, 8(3):35–61, 2005. doi: 10.21314/jcf.2005.136.

[19] C. Huré, H. Pham, and X. Warin. Deep backward schemes for high-dimensional nonlinear PDEs. *Mathematics of Computation*, 89(324):1547–1579, 2020. doi: 10.1090/mcom/3514.

[20] Y. Jiang and J. Li. Convergence of the deep BSDE method for FBSDEs with non-Lipschitz coefficients. *Probability, Uncertainty and Quantitative Risk*, 6(4):391, 2021. doi: 10.3934/puqr.2021019.

[21] D. P. Kingma and J. Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, 2015.

[22] R. Lord, R. Koekkoek, and D. van Dijk. A comparison of biased simulation schemes for stochastic volatility models. *Quantitative Finance*, 10(2):177–194, 2010. doi: 10.1080/14697680802392496.

[23] B. Negyesi and C. W. Oosterlee. A deep BSDE approach for the simultaneous pricing and delta-gamma hedging of large portfolios consisting of high-dimensional multi-asset Bermudan options. *SIAM Journal on Financial Mathematics*, 17(3):1333–1371, 2026. doi: 10.1137/25M173507X.

[24] É. Pardoux and S. Peng. Adapted solution of a backward stochastic differential equation. *Systems & Control Letters*, 14(1):55–61, 1990. doi: 10.1016/0167-6911(90)90082-6.

[25] J. Ruf and W. Wang. Neural networks for option pricing and hedging: a literature review. *The Journal of Computational Finance*, 2020. doi: 10.21314/jcf.2020.390.